

Bulls and Cows: lead the build

WEEK 2 / 1 OF 3 / Rules and original Python solution / Suggested time: 10 minutes

Companion to **bulls_and_cows.py**; the game notebook contains the same solution. Suggested total walkthrough: 25 minutes. Start with four distinct ASCII digits in every guess.

Explain the rules with a concrete example

The secret is **4271**. A **bull** is a correct digit in the correct position. A **cow** is a correct digit in a different position. Guess **1234** gives **1 bull and 2 cows**. Four bulls wins.

Ask before coding: Why store the secret as a string? **We compare characters by position; input() also returns a string. Strings preserve leading zeros.**

Original solution | unchanged logic

```
secret = "4271"

print("Welcome to Bulls and Cows!")
print("Guess the 4-digit number.")

while True:

    guess = input("Your guess: ")

    bulls = 0
    cows = 0

    for i in range(4):

        if guess[i] == secret[i]:
            bulls = bulls + 1

        elif guess[i] in secret:
            cows = cows + 1

    print("Bulls:", bulls)
    print("Cows:", cows)

    if bulls == 4:
        print("You win!")
        break
```

Build in stages: First read and print a guess. Then add the counters and for loop. Test scoring once. Finally add while True and the win condition.

Name the roles: secret is fixed state; guess changes each round; i is a position; bulls and cows count matches for the current guess.

Trace the loops aloud

WEEK 2 / 2 OF 3 / Execution order and prediction / Suggested time: 8 minutes

One complete round | secret = "4271", guess = "1234"

i	guess[i]	secret[i]	Result	Totals B / C
0	1	4	Wrong place; 1 is in secret	0 / 1
1	2	2	Exact match: bull	1 / 1
2	3	7	No match	1 / 1
3	4	1	Wrong place; 4 is in secret	1 / 2

Explain the control flow

1. while True starts another round until break ends it.
2. input pauses and returns a string.
3. Reset both counters for this new guess.
4. range(4) provides indices 0, 1, 2, 3.
5. Compare the characters at index i; otherwise test membership in secret.
6. Print totals after the for loop has completed.
7. Four bulls triggers the win message and exits the while loop.

Key question: Why elif instead of a second if? **An exact match is also present in secret. A second independent if would count a bull again as a cow.**

Key question: Why reset counters inside while? **Otherwise totals accumulate across guesses and stop describing the current round.**

Key question: Which loop does break exit? **The while loop: the for loop has already finished, and the win check is inside while.**

Predict before running | distinct-digit guesses

Guess	Bulls	Cows	Teaching point
4271	4	0	Exact solution; game ends
1724	0	4	Every digit is present, all displaced
1234	1	2	Mixture of exact, displaced and absent
4567	1	1	4 is exact; 7 is displaced
8903	0	0	No digits appear in secret

Bridge back to the intro notebook

This solution combines strings, zero-based indexing, ==, membership with in, if/elif, and counters. The two loop types are the new central idea: **for** handles a known set of positions; **while** handles an unknown number of guesses.

Quick learner task: Replace bulls = bulls + 1 with bulls += 1. Ask what changes in the output. **Nothing; this is a shorter way to increment the counter.**

Input checks and teaching pitfalls

WEEK 2 / 3 OF 3 / Limitations and extension / Suggested time: 7 minutes

Be explicit about what the original code assumes

The original program has no input validation. It is a simple solution for four distinct digits, with the fixed secret 4271. State that assumption before the live demonstration.

Input	Original behavior
12	IndexError when i reaches 2; the game stops.
42710	Only the first four characters are checked; it incorrectly wins.
abcd	Accepted as input; produces 0 bulls and 0 cows.
2222	Produces 1 bull and 3 cows, reusing the one secret digit 2.

Small extension | validate before counting

Replace the original input line with this block inside while True. Keep the existing counters and scoring code immediately after it, at their current indentation.

```
guess = input("Your guess: ").strip()

if len(guess) != 4:
    print("Enter exactly four digits.")
    continue

if not all(ch in "0123456789" for ch in guess):
    print("Use digits 0-9 only.")
    continue

if len(set(guess)) != 4:
    print("Use four different digits.")
    continue
```

Explain only what is new: strip() removes outer whitespace. all(...) requires every character to pass the digit check. set removes duplicates, so four unique characters give a set of length four. Each continue restarts the while loop before scoring.

Scope: This validation matches the unique digits in secret 4271. If repeated digits are allowed in a future version, membership alone is insufficient: scoring must track how many copies remain unmatched.

Quick verification after the extension

Try **12**, **42710**, **abcd** and **2222**: each should show a validation message and request another guess. Try **1234**: expect 1 bull, 2 cows. Try **4271**: expect 4 bulls, 0 cows and a win.

If the class finishes early

Add an attempts counter outside while and increment it only after validation. Ask learners to show the number of valid guesses in the win message. Another extension is a maximum number of attempts.

Source: bulls_and_cows.py and bulls_and_cows.ipynb in sessions/week_2/scripts/game. The validation block is a suggested extension; the source files have not been changed.